
Génie Logiciel

CER | Docker

Mardi 17 février 2026

TABLE DES MATIÈRES

Étude de cas : Docker	2
AAV	2
Mots clés	3
Contexte	3
Problématique	3
Contraintes	3
Généralisation	3
Livrables	3
Pistes de solutions	4
Plan d'actions	4
Démarche : Docker	5
Définition: Mots clés	5
Corbeille check	7
Workshop	10
Analyse et résolution : Docker	11
Conclusion	11
Capitalisation	12
Ressources	13

ÉTUDE DE CAS : DOCKER

AAV

- 2 Connaître / comprendre le mécanisme de communication des sockets
- 2 Connaître / comprendre le principe de fonctionnement de l'architecture Client-Serveur
- 5 Transmission de flux
- 5 Développer une IHM Client-Serveur manipulant des sockets en C#

MOTS CLÉS

- Conteneur Docker
- Unité de traitement
- Ports IP
- Architecture client-serveur
- Socket
- Flux de données

CONTEXTE

Le client souhaite avoir une vision sur l'état d'avancement de la fabrication de ses chaînes de production.

PROBLÉMATIQUE

Comment envoyer les données à distance au client et héberger l'application de visualisation ?

CONTRAINTES

- Serveur Docker en DMZ
- Code fourni

GÉNÉRALISATION

- Socket C#
- Conteneurisation
- IPC

LIVRABLES

- Code complet
- Plan d'utilisation Docker

PISTES DE SOLUTIONS

- Utiliser une architecture client-serveur avec des sockets pour envoyer les données à distance
- Conteneuriser l'application de visualisation avec Docker pour faciliter son déploiement et son hébergement
- Mettre en place une communication sécurisée entre le serveur Docker en DMZ et le client pour protéger les données transmises

PLAN D'ACTIONS

- Étudier les concepts de conteneurisation avec Docker et les mécanismes de communication avec les sockets
- Étudier les différentes options de communication sécurisée pour protéger les données transmises entre le serveur Docker et le client
- Étudier le fonctionnement de Socket et comment l'utiliser pour envoyer des données à distance en C#
- Étudier le code fourni pour comprendre les données à transmettre et les besoins de l'application de visualisation
- Configurer un serveur Docker en DMZ pour héberger l'application de visualisation
- Conteneuriser l'application de visualisation avec Docker
- Mettre en place la communication sécurisée entre le serveur Docker et le client pour envoyer les données à distance
-

DÉMARCHE : DOCKER

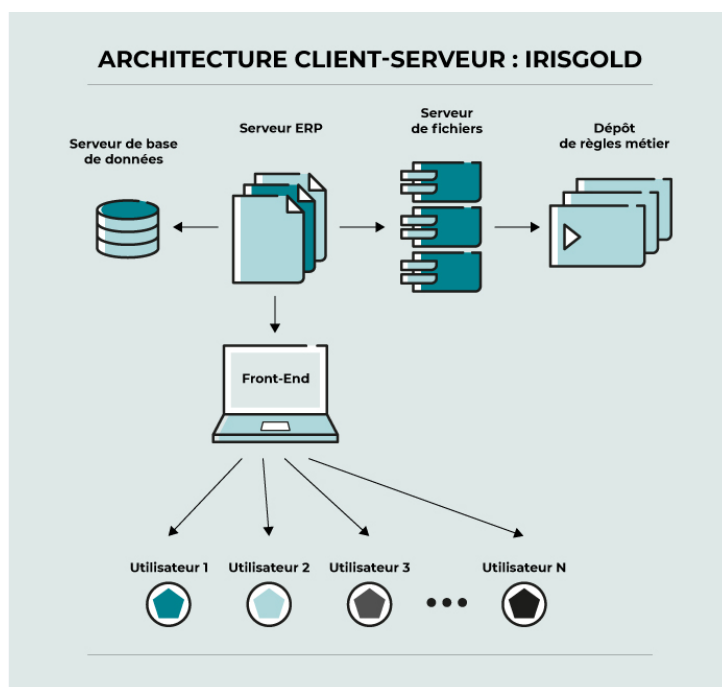
DÉFINITION : MOTS CLÉS

Conteneur Docker: Un conteneur Docker est une unité de traitement légère et portable qui encapsule une application et toutes ses dépendances, permettant ainsi de garantir que l'application fonctionne de manière cohérente dans différents environnements.

Unité de traitement: Une unité de traitement est une ressource informatique qui peut exécuter des tâches ou des processus. Dans le contexte de Docker, un conteneur est considéré comme une unité de traitement car il peut exécuter une application de manière isolée.

Architecture client-serveur: Une architecture client-serveur représente l'environnement dans lequel des applications de machines clientes communiquent avec des applications de machines de type serveurs.

L'exemple classique est le navigateur Web d'un client qui demande (on parle de "requête") le contenu d'une page Web à un serveur Web qui lui renvoie le résultat (on parle de "réponse").



Socket: Un socket est un point de communication utilisé pour établir une connexion entre deux processus sur un réseau.

Méthode	Description
Socket.Bind	Lie la socket à un point de communication, renvoie une erreur si le port spécifié par le point de communication est déjà utilisé par un autre processus
Socket.Listen	Place en file d'attente (queue) toutes les connexions entrantes
Socket.Accept	Lit la file d'attente et accepte les connexions entrantes. Accept est bloquant, l'exécution du code est bloquée jusqu'à ce qu'accept détecte une connexion entrante dans la file d'attente
Socket.Select	Permet entre autres de vérifier si une socket connectée tente d'écrire quelque chose. En passant un tableau de connexion en paramètre à select, après l'exécution de select, le tableau ne contient plus que les sockets ayant des données prêtes à être lues
Socket.Poll	Permet, un peu comme select, de détecter si une socket est prête à être lue ou écrite. À l'inverse de select, elle permet aussi de détecter si une connexion est terminée
Socket.Receive	Permet de réceptionner les données qui ont été écrites sur la socket
Socket.Send	Permet d'écrire des données sur une socket
Socket.Connect	Permet de se connecter à une socket

Important: Manipuler des sockets bruts demande de bien gérer l'asynchronisme (Task, async/await) pour éviter de bloquer votre application pendant que vous attendez une réponse du réseau.

Flux de données: Un flux de données est une séquence de données qui est transmise d'un point à un autre. Dans le contexte de Docker, les flux de données sont utilisés pour envoyer des informations entre les conteneurs et les applications externes.

Ports IP: Un port IP est un point de communication utilisé pour identifier une application ou un service spécifique sur un réseau. Dans le contexte de Docker, les ports IP sont utilisés pour permettre la communication entre les conteneurs et les applications externes.

CORBEILLE CHECK

Question: Compléter le code

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

namespace Client {
    class Program {
        static void Main(string[] args) {

            IPEndPoint ipHost = Dns.GetHostEntry(Dns.GetHostName());
            IPAddress ipAddr = ipHost.AddressList[0];
            IPEndPoint localEndPoint = new IPEndPoint(ipAddr, 1200);

            Socket sender = new Socket(ipAddr.AddressFamily, SocketType.Stream,
            ProtocolType.Tcp);

            try {
                sender.Connect(localEndPoint);

                Console.WriteLine("Socket connecté à -> {0}");
                Console.WriteLine(sender.RemoteEndPoint.ToString());

                byte[] messageSent = Encoding.ASCII.GetBytes("Test Client<EOF>");
                int byteSent = sender.Send(messageSent);
                byte[] messageReceived = new byte[1024];
                int byteRecv = sender.Receive(messageReceived);

                Console.WriteLine("Message du Serveur -> {0}",
                Encoding.ASCII.GetString(messageReceived, 0, byteRecv));

                sender.Shutdown(SocketShutdown.Both);
                sender.Close();

            } catch (ArgumentNullException ane) {
                Console.WriteLine("ArgumentNullException : {0}", ane.ToString());
            } catch (SocketException se) {
                Console.WriteLine("SocketException : {0}", se.ToString());
            } catch (Exception e) {
                Console.WriteLine("Unexpected exception : {0}", e.ToString());
            }
        }
    }
}
```


}

Question: Compléter le code

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

namespace Server {
    class Program {
        static void Main(string[] args) {

            IPEndPoint ipHost = Dns.GetHostEntry(Dns.GetHostName());
            IPAddress ipAddr = ipHost.AddressList[0];
            IPEndPoint localEndPoint = new IPEndPoint(ipAddr, 11111);

            Socket listener = new Socket(ipAddr.AddressFamily, SocketType.Stream,
            ProtocolType.Tcp);

            try {
                listener.Bind(localEndPoint);
                listener.Listen(10);

                while (true) {
                    Console.WriteLine("Waiting connection ... ");
                    Socket clientSocket = listener.Accept();

                    byte[] bytes = new Byte[1024];
                    string data = null;

                    while (true) {
                        int numByte = clientSocket.Receive(bytes);
                        data += Encoding.ASCII.GetString(bytes, 0, numByte);
                        if (data.IndexOf("<EOF>") > -1)
                            break;
                    }
                    Console.WriteLine("Text reçu -> {0} ", data);

                    byte[] message = Encoding.ASCII.GetBytes("Test Server");
                    clientSocket.Send(message);

                    clientSocket.Shutdown(SocketShutdown.Both);
                    clientSocket.Close();
                }
            } catch (Exception e) {
                Console.WriteLine(e.ToString());
            }
        }
    }
}
```

WORKSHOP

ANALYSE ET RÉOLUTION : DOCKER

CONCLUSION

CAPITALISATION

AAV	Commentaire	Statut
Définir Thread, Multi-threading, Mécanismes d'IPC, Ressource critique / Section critique, inter-blocage	Les définitions de Thread, Multi-threading, IPC, Ressource critique, Section critique et inter-blocage sont fournies avec des exemples et explications claires.	Acquis
Citer, Décrire et expliquer les principaux problèmes liés à la section critique	Les problèmes comme l'interblocage, la famine, l'inversion de priorité sont identifiés, expliqués et illustrés avec des solutions proposées.	Acquis
Citer, Décrire et expliquer les principaux mécanismes d'IPC et les organiser en catégories	Les mécanismes d'IPC (sémaphores, mutex, etc.) sont présentés, catégorisés et expliqués avec des cas d'utilisation.	Acquis
Implémenter les mécanismes de synchronisation en C#	Des exemples de code C# illustrent l'utilisation de Thread, ThreadPool, lock, Semaphore, Mutex pour la synchronisation.	Acquis

RESSOURCES

-